

Neural Stochastic Differential Equations: An introduction

Enrico Savorgnan
University of Trieste

June 11, 2026

Abstract

This article is a review of Neural Stochastic Differential Equations (Neural SDEs), mainly based on the diffusion-limit formulation of Tzen and Raginsky [2] and on the scalable gradient methods of Li et al. [9]. It summarizes how Deep Latent Gaussian Models can be interpreted as discrete approximations of Neural SDEs, how variational inference can be moved from finite-dimensional latent variables to Wiener space, and how Girsanov’s theorem turns an optimization over path measures into an optimization over drift perturbations. The final part reviews why differentiating through SDE solvers is difficult and how stochastic adjoint methods address this problem.

1 Introduction

This article reviews the topic of Neural Stochastic Differential Equations (Neural SDEs). The exposition follows two main references: Tzen and Raginsky’s work on Neural SDEs as Deep Latent Gaussian Models in the diffusion limit [2], and Li et al.’s work on scalable gradients for SDE models [9]. The goal is not to introduce a new model, but to organize the main mathematical and computational ideas behind this framework.

Neural SDEs combine latent-variable generative modelling with stochastic continuous-time dynamics. Instead of evolving through a finite sequence of deterministic layers only, the latent state follows a random path driven by Brownian noise, a drift term, and a diffusion term. Neural networks parameterize these coefficients, allowing the model to learn continuous-time stochastic transformations of a latent distribution.

The first part of the review explains how Deep Latent Gaussian Models (DLGMs) can be written as time-inhomogeneous discrete-time Markov chains. Since their likelihood requires marginalizing over many Gaussian latent variables, variational inference and the Gibbs variational principle naturally appear. Taking the diffusion limit of this discrete construction leads to an Ito SDE and motivates the Neural SDE formulation.

The second part moves the variational problem to Wiener space. There, direct optimization over probability measures is impractical, and Girsanov’s theorem provides the key reparametrization: an admissible change of measure can be represented by adding a drift to a reference Wiener process. This yields the variational free-energy objective used later in the article.

The last part focuses on differentiation. Training Neural SDEs requires gradients through stochastic dynamics, where the sampled Brownian path matters. The review therefore discusses direct discretization, sensitivity equations, and the stochastic adjoint approach of Li et al., which uses Stratonovich dynamics, invertible stochastic flows, and backward adjoint systems to obtain scalable gradient computations.

The article is organized as follows. Section 2 reviews DLGMs and their diffusion-limit interpretation. Section 3 develops variational inference for Neural SDEs on Wiener space. Section 4 discusses direct differentiation strategies and their limitations. Section 5 introduces stochastic adjoint methods.

2 DLGMs are discrete Neural SDE

The goal of Deep Latent Gaussian Models (DLGMs), as proposed by Rezende et al. [1], consists into the estimation of the probability density function (PDF) of the observed variables, conditioned on a sufficiently large sample size.

Formally, DLGMs can be seen as time-inhomogeneous discrete-time Markov chains [1] :

$$X_0 = Z_0 \tag{1}$$

$$X_t = X_{t-1} + \mu_t(X_{t-1}; \theta) + \sigma_t(X_{t-1}; \theta)Z_t = f_\theta(Z_t; \theta) \quad t = 1, \dots, k \tag{2}$$

$$Y \sim \mathbb{P}[\cdot | X_k], \tag{3}$$

where $Y \in \mathbb{R}^d$ is the observed variable, $X_0, \dots, X_k \in \mathbb{R}^d$ the latent variables, $Z_0, \dots, Z_k \sim \mathcal{N}(0, \mathbb{I}_d)$ are i.i.d. samples in $\mathbb{R}^d$, $\mu_1, \dots, \mu_k$ and $\sigma_1, \dots, \sigma_k$ are parametric nonlinear transformations.

The goal of the framework is to capture the probability density of Y , computed marginalizing w.r.t the random variables $Z_0, \dots, Z_k$ the joint distribution:

$$\mathbb{P}_\theta[Y, Z_0, \dots, Z_k] = \mathbb{P}[Y | f_\theta(Z_0, \dots, Z_k)] \phi(Z_0) \dots \phi(Z_k), \tag{4}$$

ϕ indicating the standard gaussian density in $\mathbb{R}^d$: $(2\pi)^{-d/2} \exp -\frac{1}{2}||Z_i||^2$.

Typically the integration is intractable, so that the common escape is to exploit the Jensen’s inequality and optimize the *Evidence Lower Bound* (ELBO). Given $\nu(z_0, \dots, z_k | y)$ over the latent variables, the ELBO reads as:

$$\log \mathbb{P}_\theta[y] \geq \mathbb{E}_\nu [\log \mathbb{P}_\theta[y | f_\theta(z_0, \dots, z_k)]] - \text{KL}(\nu(z_0, \dots, z_k | y) || \eta(z_0, \dots, z_k)), \tag{5}$$

where $\eta(z_0, \dots, z_k) = \prod_{i=0}^k \phi(z_i)$ is the Gaussian prior measure induced by the independent latent variables.

which can be rephrased by using the Gibbs variational principle [3] as:

$$-\log \mathbb{E}_\eta[\exp(-G(z_0, \dots, z_k))] = \inf_{\nu \in \mathcal{P}(\Omega)} \{ \text{KL}(\nu||\eta) + \mathbb{E}_\nu[G(z_0, \dots, z_k)] \}, \quad (6)$$

where $\mathcal{P}(\Omega)$ denotes the set of all the probabilities measures defined over the domain Ω , i.e., in our case, $(\mathbb{R}^d)^{k+1}$.

This formulation is more convenient for our purposes, since it will allow a nice derivation that we will cover in the following.

The Gibbs variational principle for probability measures on $\mathbb{W}$ asserts that the marginal negative log-likelihood of the observations can be bounded exactly by an infimum over all absolutely continuous probability measures :

$$-\log p_\theta(y) = \inf_{\nu \in \mathcal{P}(\mathbb{W})} \{ \text{KL}(\nu||\mu) - \int_{\mathbb{W}} \log p(y|[f_\theta(w)]1) \nu(dw) \} \quad (7)$$

where $\text{KL}(\nu||\mu)$ is the Kullback-Leibler divergence between the approximate posterior measure ν and the prior Wiener measure μ on the path space.

To bring a DLGM to the *diffusion limit* means to consider the case for $k \rightarrow \infty$ and $\Delta t \rightarrow 0$. In such scenario, one can rewrite the Markov Chain as an Ito Stochastic Differential Equations (SDE):

$$dX_t = \mu(X_t, t)dt + \sigma(X_t, t)dW_t \quad t \in [0, 1], dW_t \sim \mathcal{N}(0, dt) \quad (8)$$

Crucially, in order to formulate correctly the Ito SDE, it should hold that $\mu(\cdot) \propto \Delta t$ and $\sigma(\cdot) \propto \sqrt{\Delta t}$.

In a neural scenario, we can think of μ, σ (called, respectively, the *drift* and the *diffusion*) to be both generated by neural networks. Y is still generated from $\mathbb{P}[\cdot|X_1]$.

The framework here introduced takes in literature the name of *Neural SDE*.

Tzen and Raginsky [2] showed that setting $\sigma \equiv \mathbb{I}_d$ is expressive enough to approximate samples from any sufficiently smooth target distribution, thus limiting the network to only learn the drift. For “sufficiently smooth” we indicate here target distributions $q(x)$ whose *Radon-Nikodym derivative* w.r.t. to the standard gaussian measure can be reconstructed by neural networks.

In a glimpse, the Radon-Nikodym derivative is a function g , usually written in a derivative-like form such as $\frac{d\alpha}{d\beta}$, such that given two probability measures α, β defined over Ω it holds that:

$$\forall A \subset \Omega, \alpha(A) = \int_A g d\beta \quad (9)$$

In practice, it says that there exists a map from β to α , and the map is driven by g itself. In deterministic scenarios, the existence of the map is guaranteed by Brenier’s theorem [4], provided that both the prior and the target have a finite second moment.

Said $q(x) = f(x)\phi_d(x)$ to be the complex target probability density to map from a standard Gaussian prior distribution at time $t = 0$, the exact mapping can be achieved through the following SDE:

$$dX_t = \nabla \log Q_{1-t}f(X_t)dt + dW_t, \quad X_0 = 0, \quad t \in$$

where $Q_t f(x) := \mathbb{E}_{Z \sim \phi_d}[f(x + \sqrt{t}Z)]$ defines the so-called *heat semigroup operator* applied to the Radon-Nikodym derivative $f(x)$.

The critical drift term, $\nabla \log Q_{1-t}f(X_t)$, is known in the literature as the *Föllmer drift*.

The expressiveness of Neural SDEs stems from the fact that a sufficiently wide deep neural network can parameterize this Föllmer drift directly.

Replacing the exact expectation $Q_{1-t}f(x)$ with a Monte Carlo estimate and $f(\cdot)$ with a neural network approximation $\hat{f}(\cdot; \theta)$, the model can approximate the optimal transport map between the prior and target distributions. Crucially, the neural network approximating the Föllmer drift takes both the spatial variable X_t and the temporal variable t as continuous inputs.

This means its weight parameters θ do not need to explicitly depend on time, providing a massive parameter efficiency advantage over discrete DLGM, which typically require distinct weight matrices θ_i for every single layer i in the Markov chain.

The continuous formulation allows a single set of shared weights to govern the entire evolutionary trajectory, conditioned continuously on time t .

3 Variational Inference for Neural SDE

After having introduced the Neural SDE, the next goal is to adapt the variational framework to the stochastic setting, so to define a training objective.

The latent space is, in this new scenario, provided by the *Wiener space* $\mathbf{W}$, meaning the space of continuous paths $w : [0, 1] \rightarrow \mathbb{R}^d$ whose primitive is the Wiener Process $W = \{W_t\}_{t \in [0, 1]}$.

The usual discrete approach requires Z_i to be i.i.d.: here, it is required for each increment $dW_t = W_t - W_s$ to be independent to the previous ones.

We can define a probability measure $\mathbb{Q}(dW)$ over the Wiener process too; this measure is such that for each $0 < t_1 < t_2, \dots \leq 1$, $W_{t_i} - W_{t_{i-1}} \sim \mathcal{N}(0, (t_i - t_{i-1})\mathbb{I}_d)$. Bichteler [5] showed that if the neural mappings for μ and σ are continuous, than they are measurable, and:

$$f_t(W; \psi) = \int_0^t \mu(f_s(W; \psi), s; \theta) ds + \int_0^t \sigma(f_r(W; \psi), r; \phi) dW_r, \quad (10)$$

with θ, ϕ, ψ parameters, and $f_{(\cdot)}$ the mapping from W_0 to $W_{(\cdot)}$. To simplify the notation, we will mark all the parameters with a single letter θ . Finally, we can derive a stochastic formulation, equivalent to the discrete case in 4, as:

$$\mathbb{P}[dY, dW] = \mathbb{P}[Y | f_1(W, \theta) \mathbb{Q}[dW] dY], \quad (11)$$

and our target is, once again, the marginal distribution $\mathbb{P}_\theta[Y] = \int_{\mathbf{W}} \mathbb{P}[Y | f_1(W, \theta) \mathbb{Q}[dW]]$.

In this scenario, the ELBO reads (6) as:

$$-\log \mathbb{P}_\theta[Y] = \inf_{\nu \in \mathcal{P}(\mathbf{W})} \left\{ \text{KL}(\nu || \mathbb{Q}) - \int_{\mathbf{W}} \log \mathbb{P}_\theta[Y | f_1(W, \theta)] \nu(dW) \right\} \quad (12)$$

Eq.12 is difficult to optimize directly, since the infimum is taken over probability measures on the path space $\mathbf{W}$. A standard way to make this problem tractable is to use *Girsanov's theorem* [6]. In the present setting, the theorem states that any measure ν on $\mathbf{W}$ that is absolutely continuous with respect to the reference Wiener measure $\mathbb{Q}$ can be represented by adding an adapted drift to a standard Wiener process W . Equivalently, the coordinate process $Z \sim \nu$ can be obtained from $W \sim \mathbb{Q}$ through a drift perturbation; this change of measure is one-to-one (often, bijective) under

the usual integrability assumptions and yields a finite KL-divergence with a closed-form expression. Consequently, we can write

$$Z_t = \int_0^t u_s ds + W_t, \quad t \in [0, 1] \quad (13)$$

so that $Z = \{Z_t\}_{t \in [0,1]} \sim \nu$, and

$$\text{KL}(\nu || \mathbb{Q}) = \mathbb{E}_{\mathbb{Q}} \left[\frac{1}{2} \int_0^1 \|u_t\|^2 dt \right] \quad (14)$$

In our case, the standard Wiener process W is the one deriving from Eq. 10 as $f_t(W; \theta)$. The theorem allows for a nicer reparametrization of Eq. 12:

$$-\log \mathbb{P}_{\theta}[Y] = \inf_u \mathbb{E}_{\mathbb{Q}} \left\{ \frac{1}{2} \int_0^1 \|u_t\|^2 dt - \log \mathbb{P}_{\theta}[Y | Z_{(\cdot)}] \right\}, \quad (15)$$

where $Z_{(\cdot)}$ is in the form of Eq. 13.

Moreover, since u is a deterministic drift, it can be safely mapped using neural networks.

Finally, notice how in the last formulation Eq.15, there is no need for a network that has to learn the diffusion term!

Concretely, said $\tilde{\mu}(Y, t; \beta)$ the neural approximation of the process u , the final object (the *variational free energy*) to optimize becomes:

$$\mathcal{F}_{\theta, \beta}(y) = \mathbb{E} \left[\frac{1}{2} \int_0^1 \|\tilde{\mu}(y, t; \beta)\|^2 dt - \log \mathbb{P}_{\theta}[Y | Z_{(\cdot)}] \right] \quad (16)$$

4 Automatic Differentiation for Neural SDE

The optimization of Eq.16 is far to be easy to compute.

While calculating gradients is, in case of well-behaving mappings, pretty straightforward, the “back-propagation” of the error is not so simple.

Said

$$Z_t^{\theta, \beta} = Z_0 + \int_0^t \mu(Z_s^{\theta, \beta}, s; \theta) ds + \int_0^t \tilde{\mu}(y, s; \beta) ds + \int_0^t \sigma(Z_s^{\theta, \beta}, s; \theta) dW_s, \quad (17)$$

as derived from 13, the gradients of the first righthand-sided term in Eq.16 are (in case of sufficient well-behaved functions):

$$\frac{\partial}{\partial \beta} \mathbb{E}[\log \mathbb{P}_{\theta}[y | Z_{(\cdot)}]] = \mathbb{E} \left[\frac{\partial}{\partial Z} \log \mathbb{P}[y | Z_1^{\theta, \beta}] \frac{\partial}{\partial \beta} Z_1^{\theta, \beta} \right] \quad (18)$$

$$\frac{\partial}{\partial \theta} \mathbb{E}[\log \mathbb{P}_{\theta}[y | Z_{(\cdot)}]] = \mathbb{E} \left[\frac{\partial}{\partial Z} \log \mathbb{P}[y | Z_1^{\theta, \beta}] \frac{\partial}{\partial \beta} Z_1^{\theta, \theta} \right] \quad (19)$$

As said, doing automatic differentiation is not an easy task. In Neural ODE the solution was to use *adjoint methods*, i.e., to introduce a “backward” ODE, efficiently propagating the gradients by using simply an ODE solver.

The same method cannot, at least from a naïve point of view be exploited for the stochastic version.

Indeed, while stochastic adjoint methods exist, they cannot be used in practice since they rely on PDEs, thus are very slow.

Tzen and Raginsky [2] proposed two different ways to solve the problem.

The first approach consists into using the Euler-Maruyama algorithm to simulate the diffusion process, and then build the computational graph using the Euler recursion, and computing gradients using reverse-mode AD:

$$\hat{X}_{t_{i+1}}^{\theta, \beta} = \hat{X}_{t_i}^{\theta, \beta} + h_{i+1}(\mu(\hat{X}_{t_i}^{\theta, \beta}, t_i; \theta) + \tilde{\mu}(y, t_i; \beta)) + \sqrt{h_{i+1}}\sigma(\hat{X}_{t_i}^{\theta, \beta}, t_i; \theta)Z_{i+1}, \quad (20)$$

given $h_{i+1} = t_{i+1} - t_i$.

The problem of this approach is that explicit temporal discretization compromises the continuous-time advantages of the analytical model, fundamentally reducing the system to a highly parameterized, discrete DLGM. Importantly, because reverse-mode automatic differentiation requires a backward pass, every intermediate activation state $\hat{X}_{t_i}$ and every sampled noise vector Z_{i+1} must be cached in memory.

Consequently, the memory complexity scales linearly with the number of time steps $\mathcal{O}(L)$.

If a high-resolution adaptive solver requires thousands of steps to maintain numerical stability, this approach quickly exceeds computational memory limits.

An alternative solution, inspired by adjoint algorithmic differentiation for Monte Carlo sensitivities, consists into a direct computation of the derivatives, thanks to the use of SDE solvers:

$$\frac{\partial}{\partial \beta^i} X_t = \int_0^t \left(\frac{\partial}{\partial x} \mu_s \frac{\partial}{\partial \beta^i} X_s + \frac{\partial}{\partial \beta^i} \tilde{\mu}_s \right) ds + \sum_{k=1}^d \int_0^t \frac{\partial}{\partial x} \sigma_{s,k} \frac{\partial}{\partial \beta^i} X_s dW_s^k \quad (21)$$

$$\frac{\partial}{\partial \theta^j} X_t = \int_0^t \left(\frac{\partial}{\partial \theta^j} \mu_s + \frac{\partial}{\partial x} \mu_s \frac{\partial}{\partial \theta^j} X_s \right) ds + \sum_{k=1}^d \int_0^t \left(\frac{\partial}{\partial \theta^j} \sigma_{s,k} + \frac{\partial}{\partial x} \sigma_{s,k} \frac{\partial}{\partial \theta^j} X_s \right) dW_s^k \quad (22)$$

The drawback of this approach is that it has an high computational complexity, despite having a little storage overhead.

Indeed, it operates in $\mathcal{O}(1)$ memory because it does not require an unrolled backward pass, but in $\mathcal{O}(nd^2)$ time, as it requires pushing massive Jacobian matrices forward at every integration step.

Both the methods may also require Monte Carlo expectations to get estimates.

5 Stochastic Adjoint Methods

The adjoint sensitivity method is an elegant approach used in control theory [7] and then applied to machine learning as an efficient way to compute gradients with respect to parameters in Neural ODE [8]. The general idea of the method consists into adding to the system an ODE that runs “backward in time”, i.e., starts from the ending time t_1 and ends at the start of the process t_0 . This backward flow can be used to track the weights of each parameters, i.e., the derivative of loss w.r.t the parameters.

Moreover, the method works in $\mathcal{O}(1)$ memory and linearly on the number of layers in time, thus being extremely scalable compared to the others proposed in Section 4.

The adjoint sensitivity method as it is used in Neural ODE cannot be exploited in Neural SDE: indeed, it has no meaning to run an SDE “backward in time”, nor how can we “correctly” reconstruct the forward trajectory from the backward without storing the sampled noise, which are fully recovered from a simple seed thanks to the use of pseudo-random number generators and of a particular data structure called *virtual brownian tree*, whose details are out of the scope of this review.

The idea of the *stochastic adjoint process* introduced by [9] is to compute deterministically the gradient of the loss w.r.t. the given initial state x_0 , given a complete realization of a Wiener Process $\{W_t\}_{t \in [0,1]}$.

Obviously, computing the exact value of the gradient is intractable (as we will show, it requires to compute integrals over time), so one should recover to numerical approximation. However, if the approximation is good enough, the whole system, adjoints included, can be reconstructed using simple ingredients.

Li et al. [9] noticed how by using a Stratonovich SDE formulation, i.e.,

$$\phi_{s,t}(X_0) := X_t = X_0 + \int_0^t \mu(X_s, s; \theta) ds + \int_0^t \sigma(X_r, r) \circ dW_r, \quad (23)$$

if both the drift and the diffusion terms are well-behaving, meaning that $\mu, \sigma \in \mathcal{C}^{\infty,1}$ w.r.t. state and time respectively, the SDE admits a unique solution.

Importantly, the solution is invertible and admits a *backward flow* $\check{\psi}_{s,t} := \phi_{s,t}^{-1}$, which itself is the solution of a Stratonovich backward SDE:

$$\psi_{s,t}(x) = x - \int_s^t \mu(\psi_{u,t}(x), u) du - \int_s^t \sigma(\psi_{u,t}(x), u) \circ d\check{W}_u \quad (24)$$

where $d\check{W}_z$ defines the backward noise, i.e., $W_z - W_{z+1}$.

Notice the similarity between Eq. 23 and Eq. 24, due to the symmetry of Stratonovich SDEs. This symmetry is of extreme relevance, allowing, as we will see, the definition of backward differential flows and, thus, a stochastic version of the adjoint method.

Defined $J_s(X) := \nabla_X \psi_{s,t}(X)$ and its inverse $K_s := J_s(X)^{-1} = \nabla_X \phi_{s,t}(X)$, it holds:

- Simply from 24:

$$J_{s,t}(X) = \mathbb{I}^d - \int_s^t \nabla \mu(\psi_{r,t}(X), r) J_{r,t}(X) dr - \int_s^t \nabla \sigma(\psi_{r,t}(X), r) J_{r,t}(X) \circ d\check{W}_r \quad (25)$$

or analogously:

$$dJ_{s,t}(X) = -\nabla \mu(\psi_{r,t}(X), r) J_{r,t}(X) dr - \nabla \sigma(\psi_{r,t}(X), r) J_{r,t}(X) \circ d\check{W}_r \quad (26)$$

- With simple steps shown below:

$$K_{s,t}(X) = \mathbb{I}^d + \int_s^t K_{r,t}(X) \nabla \mu(\psi_{r,t}(X), r) dr + \int_s^t K_{r,t}(X) \nabla \sigma(\psi_{r,t}(X), r) \circ d\check{W}_r \quad (27)$$

The derivation of Eq. 27 is pretty straightforward.

At first, let's consider the product $KJ = \mathbb{I}^d$, where for the seek of simplicity we refused to add also the subscripts to K and J . We can differentiate both the sides of the equation, obtaining $d(KJ) = d(\mathbb{I}^d) = 0$. Since J derives from a Stratonovich SDE, so will do K ; we can thus safely apply the standard product rule and obtain:

$$\begin{aligned} d(K)J + Kd(J) &= 0 \\ d(K) &= -Kd(J)J^{-1} \\ d(K) &= -Kd(J)K \end{aligned} \tag{28}$$

Plugging 26, it holds:

$$dK = -K(-\nabla\mu(\psi, r)Jdr - \nabla\sigma(\psi, r)J \circ d\check{W}_r)K \tag{29}$$

$$= K\nabla\mu(\psi, r)JKdr + K\nabla\sigma(\psi, r)JK \circ d\check{W}_r \tag{30}$$

$$= K\nabla\mu(\psi, r)dr + K\nabla\sigma(\psi, r)d\check{W}_r \tag{31}$$

and finally, integrating both the sides, one gets 27.

Notice also , given $\nabla(\phi(\psi(X))) = \nabla X$, it holds that

$$\begin{aligned} \nabla(\phi(\psi(X))) &= \mathbb{I} \\ \nabla\phi(\psi(X))\nabla\psi(X) &= \mathbb{I} \\ \nabla\phi(\psi(X))J_{s,t}(X) &= \mathbb{I} \\ J_{s,t}(X)^{-1} &= \nabla\phi(\psi(X)) \\ K_{s,t}(X) &= \nabla\phi(\psi(X)) \end{aligned} \tag{32}$$

Now, said $A_{s,t} := \frac{\partial}{\partial X}\mathcal{L}(\phi_{s,t}(X)) = \nabla\mathcal{L}(\phi_{s,t}(X))\nabla\phi_{s,t}(X)$, we define as the *adjoint SDE* the function:

$$\begin{aligned} B_{s,t}(X) &:= A_{s,t}(\psi_{s,t}(X)) \\ &= \nabla\mathcal{L}\nabla(\phi_{s,t}(\psi_{s,t}(X))) \\ &= \nabla\mathcal{L}K_{s,t}(X) \end{aligned} \tag{33}$$

Applying 27 to 33 one gets the following *adjoint system*, whose approximation is the objective of optimization:

$$B_{s,t}(x) = \nabla\mathcal{L}(x) + \int_s^t \nabla\mu(\psi_{r,t}(x), r)^T B_{r,t}(x) + \int_s^t \nabla\sigma(\psi_{r,t}(x), r)^T B_{r,t}(x) \circ d\check{W}_r \tag{34}$$

$$\psi_{s,t}(x) = x - \int_s^t \mu(\psi_{r,t}(x), r)dr - \int_s^t \sigma(\psi_{r,t}(x), r) \circ d\check{W}_r \tag{35}$$

The last point to discuss relates to the actual approximation of the adjoint system defined above. The most common approach, in literature, consists into exploiting *forward-backward SDE* (FBSDE) technique, falling in the forward mode discussed in Section 4.

Instead, by treating the SDE solver as a deterministic functional $\mathcal{F}(x, \{W_t\})$, and omputing the

forward path thanks to the use of another deterministic functional $\mathcal{G}(x_0, \{W_t\})$, the composition of $\mathcal{F}(\mathcal{G})$ is able in common nice scenarios to fully reconstruct the adjoints, thus bypassing the need of slow or complex solvers.

Crucially, the model parameters θ and variational parameters β are mathematically treated as augmented state variables with zero drift and zero diffusion, allowing their sensitivities to be computed simultaneously within this identical backward pass without ever instantiating full, dense Jacobian matrices.

We are still missing a proper loss to compute: maximizing the likelihood causes overfitting. The solution is exactly what we have got in Eq. 15.

We only need to define properly the new Wiener process u .

[9] proposed an elegant solution. By parameterizing both the prior and the (approximate) posterior using SDEs, and choosing for both the same diffusion mechanism, the new Wiener process $u(z, t)$ can be defined as:

$$u(z, t) = \sigma(z, t)^{-1} (h_\theta(z, t) - h_\phi(z, t)) \quad (36)$$

where z is the latent variable, σ is the (common) diffusion term and h_θ, h_ϕ the (different) drifts of the prior and the posterior, respectively.

Importantly, the adjoint method can be proved to be a SDE having no diffusion term and drift equal to $\frac{1}{2} |u(z, t)|^2$.

This model, combining neural SDE and stochastic adjoint methods is called **Latent SDE** and is the main topic of the experiments.

6 Conclusion

This review has followed the construction of Neural SDEs from the discrete latent-variable setting to the continuous stochastic setting. The first step was to interpret DLGMs as time-inhomogeneous Markov chains driven by Gaussian latent variables. Since the likelihood of the observed variable requires marginalization over all latent variables, the discrete model naturally motivates variational approximations and the use of the Gibbs variational principle.

The diffusion-limit viewpoint then replaces the finite sequence of Gaussian perturbations with a continuous Brownian path. In this limit, the latent dynamics are described by an Ito SDE whose drift and diffusion may be parameterized by neural networks. This perspective is useful because it gives a precise continuous-time interpretation of deep latent Gaussian architectures: depth becomes time, layerwise Gaussian noise becomes Brownian motion, and the learned transition maps become coefficients of a stochastic differential equation.

The variational formulation on Wiener space is the key mathematical step that makes this framework trainable. Directly optimizing over probability measures on path space is not practical. Girsanov’s theorem provides the required reparametrization: instead of searching over measures, one searches over drift perturbations of a standard Wiener process. The resulting objective, written as a variational free energy, separates a quadratic control cost from the expected negative log-likelihood of the generated path.

The remaining difficulty is computational. Gradients through SDEs cannot be treated exactly as gradients through ODEs, because the Brownian path and its discretization affect the backward computation. Discretize-then-differentiate methods such as Euler–Maruyama are direct but memory-dependent, while sensitivity-based methods require tracking the noise. Stochastic adjoint methods address this issue by using Stratonovich dynamics, invertible stochastic flows, and a backward adjoint system. This gives the conceptual route toward scalable training of Neural SDEs, which is the main computational message of the final section.

Overall, Neural SDEs can be understood as a bridge between deep latent Gaussian generative models, stochastic calculus, variational inference, and adjoint-based optimization. The framework is powerful precisely because these ingredients fit together: diffusion limits explain the model class, Girsanov’s theorem explains the variational objective, and stochastic adjoints explain how gradients can be propagated through the resulting stochastic dynamics.

References

- [1] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic Backpropagation and Approximate Inference in Deep Generative Models. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1278–1286, 2014.
- [2] Belinda Tzen and Maxim Raginsky. Neural Stochastic Differential Equations: Deep Latent Gaussian Models in the Diffusion Limit. *arXiv preprint arXiv:1905.09883*, 2019.
- [3] Paul Dupuis and Richard S. Ellis. *A Weak Convergence Approach to the Theory of Large Deviations*. Wiley, 1997.
- [4] Yann Brenier. Polar Factorization and Monotone Rearrangement of Vector-Valued Functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991.
- [5] Klaus Bichteler. *Stochastic Integration with Jumps*. Cambridge University Press, 2002.
- [6] Igor V. Girsanov. On Transforming a Certain Class of Stochastic Processes by Absolutely Continuous Substitution of Measures. *Theory of Probability & Its Applications*, 5(3):285–301, 1960.
- [7] Lev S. Pontryagin, Vladimir G. Boltyanskii, Revaz V. Gamkrelidze, and Evgenii F. Mishchenko. *The Mathematical Theory of Optimal Processes*. Interscience Publishers, 1962.
- [8] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural Ordinary Differential Equations. In *Advances in Neural Information Processing Systems*, 2018.
- [9] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable Gradients for Stochastic Differential Equations. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, pages 3870–3882, 2020.